

ASSESSMENT TOOL-2

Scenario-Based Questions

1. Cloud Architecture for a Video Streaming Platform

A media company streaming videos to millions of users needs an architecture that can handle high traffic, reduce buffering, and automatically adjust resources according to demand.

Proposed Cloud Architecture

The architecture can include:

**Users → CDN → Load Balancer → Auto-Scaling
Application Servers → Video Storage/Database**

1. Content Delivery Network (CDN)

A CDN stores cached copies of frequently accessed videos at locations closer to users.

- Reduces network latency.
- Reduces load on the main cloud servers.
- Provides faster video delivery.
- Helps minimize buffering.

2. Load Balancing

A load balancer distributes incoming user requests across multiple application servers.

For example, if one server becomes heavily loaded, new requests can be directed to less-loaded servers.

Benefits:

- Prevents server overload.
- Improves availability.
- Provides better response time.
- Supports millions of concurrent users.

3. Auto-Scaling

Auto-scaling automatically increases or decreases the number of servers based on demand.

For example:

- During a popular live event → more servers are automatically added.
- During low-demand hours → unnecessary servers are removed.

This ensures that sufficient computing resources are available without keeping maximum capacity running continuously.

4. Caching

Frequently accessed videos and metadata can be cached at the CDN and application layers.

This reduces repeated requests to the origin storage and improves streaming performance.

Cost Optimization

The company can:

- Use pay-as-you-go cloud resources.
- Automatically scale resources during low-demand periods.
- Use object storage for videos instead of expensive compute storage.
- Use CDN caching to reduce origin bandwidth costs.
- Use reserved/committed capacity for predictable workloads.
- Compress and encode videos into suitable resolutions.

Conclusion

A combination of **CDN, caching, load balancing, auto-scaling, and cloud storage** can provide smooth and reliable video streaming. Auto-scaling also helps control costs by matching resource usage with actual demand.

2. Multi-Cloud Container Deployment for a Fintech Startup

The fintech startup uses containers across multiple cloud providers but faces problems with consistency and deployment management. A container orchestration platform can provide a common management layer.

Proposed Solution

The startup can use **Kubernetes** as the container orchestration platform across different cloud providers.

Application → Container → Kubernetes Cluster → Multiple Cloud Providers

1. Container Orchestration

Kubernetes can manage:

- Container deployment
- Scheduling
- Health checks
- Service management
- Automatic restart of failed containers
- Scaling

The same container images can be deployed across different cloud environments, improving consistency.

2. Consistent Deployment

The startup should maintain application configuration and infrastructure definitions using **Infrastructure as Code (IaC)**.

Container images can be stored in a secure container registry and versioned properly.

A CI/CD pipeline can automatically:

1. Build the application.
2. Test the container.
3. Scan it for vulnerabilities.
4. Deploy the approved version to cloud clusters.

3. Service Discovery

Kubernetes provides internal service discovery. Applications can communicate using stable service names instead of depending on changing container IP addresses.

For example:

Payment Service → Authentication Service → Database Service

If a container is replaced, the service name remains the same.

4. Scaling

Kubernetes can automatically increase or decrease the number of container instances based on CPU, memory, or application metrics.

For example, during heavy payment traffic, additional instances of the payment service can be created.

Benefits of Multi-Cloud

- Reduces dependency on one cloud provider.
- Improves availability.
- Provides flexibility in choosing cloud services.
- Helps reduce vendor lock-in.
- Can provide geographic flexibility.

Risks

- Increased complexity.
- Different cloud services and networking models.
- Higher monitoring and management requirements.
- Possible data synchronization problems.
- Increased operational cost.
- Security policies must be consistently applied across clouds.

Conclusion

Using **Kubernetes, container registries, CI/CD, Infrastructure as Code, and centralized monitoring** can provide consistent application deployment across multiple clouds. However, strong security and centralized governance are required to manage the additional complexity.

3. Backup and Disaster Recovery for an E-Learning Platform

The e-learning platform experienced data loss because of accidental deletion. A cloud-native backup and disaster recovery strategy should protect data from accidental deletion, corruption, and infrastructure failures.

Proposed Architecture

Primary Database → Continuous Replication → Secondary Region

Application Data → Versioned Object Storage → Backup Storage

1. Cloud-Native Backup

Regular automated backups should be created for:

- Databases

- Application data
- Configuration files
- Important storage volumes

Backups should be stored separately from the production environment.

2. Versioning

Object storage versioning should be enabled.

If a file is accidentally deleted or overwritten, an earlier version can be restored.

For example:

**Assignment_v1 → Assignment_v2 → Accidentally Deleted
→ Restore Assignment_v2**

3. Replication

Important data should be replicated to another availability zone or region.

If the primary environment fails, the secondary environment can be activated.

RTO and RPO

Recovery Time Objective (RTO):

The maximum acceptable time required to restore the service after a failure.

Recovery Point Objective (RPO):

The maximum acceptable amount of data that can be lost, measured in time.

For example:

- RTO = 30 minutes
- RPO = 5 minutes

This means the organization aims to restore the service within 30 minutes and lose no more than approximately 5 minutes of data.

How to Achieve RTO

- Maintain a standby environment.
- Automate infrastructure deployment.
- Use replicated databases.
- Maintain ready-to-use application servers.
- Automate failover procedures.

How to Achieve RPO

- Use frequent automated backups.

- Enable database replication.
- Use continuous or near-continuous data synchronization.
- Enable object versioning.

Business Continuity Steps

1. Detect the failure.
2. Identify the affected resources.
3. Stop further data corruption if necessary.
4. Activate the backup/secondary environment.
5. Restore the latest valid data.
6. Redirect users to the recovered system.
7. Verify application and data integrity.
8. Monitor the recovered environment.
9. Analyze the failure and improve the recovery plan.

Conclusion

Combining **automated backups, versioning, cross-region replication, and automated failover** provides strong protection against data loss and helps the e-learning platform continue operating during failures.

4. Hybrid Cloud Architecture for a Government Agency

The government agency needs to keep sensitive applications on-premises while using the cloud for non-sensitive workloads. A

hybrid cloud architecture is suitable because it combines private/on-premises infrastructure with public cloud resources.

Proposed Architecture

Users → Identity Provider → Secure Gateway

On-Premises:

Sensitive databases + Critical applications

Public Cloud:

Non-sensitive applications + Web services + Scalable workloads

The two environments can communicate through a **secure VPN or dedicated private connection**.

1. Secure Communication

A secure connection should be established between the government datacenter and cloud provider.

Possible methods include:

- Site-to-site VPN
- Dedicated private network connection
- Firewalls
- Network segmentation
- Intrusion detection and prevention

Only authorized traffic should be allowed between the environments.

2. Identity Management

A centralized identity management system should be used.

It should provide:

- Single Sign-On where appropriate.
- Multi-factor authentication.
- Role-Based Access Control.
- Least-privilege access.
- Centralized user management.

For example, an employee working in finance should only have access to systems required for their role.

3. Encryption

Data in transit:

Use secure protocols such as TLS when transferring information between users, cloud services, and on-premises systems.

Data at rest:

Sensitive databases and cloud storage should use encryption.

Encryption keys should be securely managed and access to keys should be restricted.

Challenges

- Different security policies between environments.
- Complex network management.
- Identity integration problems.
- Compliance and auditing requirements.
- Data synchronization difficulties.
- Higher operational complexity.
- Monitoring multiple environments.

Conclusion

A hybrid cloud can provide both **security and flexibility**. Sensitive workloads can remain on-premises while scalable, non-sensitive applications use the public cloud. Secure networking, centralized identity management, encryption, and continuous monitoring are essential.

5. Multi-Region High Availability for a SaaS Application

A SaaS provider needs its application to remain available even if an entire cloud region fails. A **multi-region architecture** can provide high availability and fault tolerance.

Proposed Architecture

Users → Global DNS/Traffic Manager → Region 1 / Region 2

Each region contains:

- Load balancer
- Application servers
- Database
- Cache
- Monitoring system

Both regions should be capable of handling production traffic.

1. Multi-Region Deployment

Deploy the application in at least two geographically separated regions.

For example:

Region A → Primary

Region B → Secondary/Standby

For stronger availability, both regions can operate in an **active-active** configuration, where both handle user traffic.

2. Load Balancing

A global traffic management service can distribute users between regions.

If Region A becomes unavailable, traffic can automatically be redirected to Region B.

3. Data Replication

Important application data should be replicated between regions.

Depending on application requirements, the provider can use:

- Synchronous replication for very strong consistency where latency allows.
- Asynchronous replication for better geographic performance.

Database replication should be monitored continuously to detect replication delays.

4. Failover

A health-check system continuously checks the availability of each region.

If Region A fails:

Region A failure → Health check detects failure → Traffic redirected → Region B serves users

Failover should be automated wherever possible.

5. Monitoring and Alerting

The SaaS provider should monitor:

- CPU and memory usage
- Application response time
- Error rates
- Database health
- Network performance
- Replication lag
- Number of active users
- Availability of each region

Alerts should be generated when predefined thresholds are exceeded.

For example, a sudden increase in error rate can trigger an alert before a complete service failure occurs.

6. Testing

The organization should regularly perform:

- Backup restoration tests
- Failover tests
- Disaster recovery drills
- Application health checks
- Load testing

This ensures that the recovery system actually works when needed.

Conclusion

A multi-region architecture with global load balancing, replicated data, automated failover, and continuous monitoring can keep a SaaS application available even when one cloud region becomes unavailable. It improves reliability and reduces the impact of regional failures.